

Forge (the InterGenOS system installer) is an ambitious, mostly well-structured piece of software that ties together the build system and package manager into a complete installation experience. It follows a classic guided TUI flow and reuses components from igos-build and pkm effectively (e.g., --root support in PackageInstaller, manifest handling, post-install hooks).

Since this code has **not yet been tested on real hardware or in a VM**, my review focuses heavily on **safety, correctness, and robustness** — especially around destructive operations like partitioning and bootloader installation. The design is pragmatic for a from-source LFS-derived distro, but it contains several high-severity risks that must be addressed before any real deployment.

I'll address your specific points first, then provide broader feedback and prioritized recommendations.

1. Disk Partitioning Safety

This is the most critical and currently weakest area.

Strengths:

- Uses wipefs -a + parted -s (scripted, non-interactive) — consistent.
- Detects EFI vs BIOS correctly via /sys/firmware/efi.
- Handles NVMe/mmcblk naming for partition devices (_partition_paths).
- Single confirmation dialog before partitioning.

Serious concerns:

- **No list of current partitions or data preview** — the user sees only a disk list with size/model, then a generic "ALL DATA WILL BE ERASED" warning. No lsblk preview of existing partitions, no "this will destroy the following filesystems" summary.
- **No final explicit double-confirmation** with the exact disk path (/dev/nvme0n1) typed back or "YES I UNDERSTAND" required. Many installers (Calamares, Debian) require this for destructive actions.
- **No dry-run or simulation mode.**
- **Hard-coded simple layout** (512 MiB ESP or 1 MiB bios_grub + rest as root/ext4). No swap, no separate /home, no LVM, no encryption option. This is acceptable for a minimal "first boot" installer but should be documented as such.

- Partition size calculations are simple and mostly correct (1 MiB start for alignment, 513 MiB ESP), but no alignment checks or parted alignment flags.

Verdict: Insufficient safeguards. The risk of accidentally wiping the wrong disk (especially in multi-disk systems or when running over SSH/serial) is too high. Add at least one more explicit confirmation screen that echoes the exact parted commands that will run and requires typing the disk name.

2. Fstab Generation

Mostly good, with sensible fallbacks.

- Uses `blkid -s UUID -o value` — correct and reliable.
- Falls back to device path if UUID extraction fails.
- Root uses defaults 1 1; ESP uses defaults 0 2 — standard and appropriate for ext4/vfat.
- No separate options for SSD trim (discard), noatime, etc. — could be added later.

Minor issues:

- No handling for swap (if added later).
- Assumes only root + optional ESP; fine for v1.
- `_get_uuid` silently returns None on failure — the fallback to device path is good, but log the fallback.

Verdict: Correct and robust enough for the simple layout.

3. Chroot Hook Execution & Virtual FS Handling

The `hooks.py` implementation follows standard practices but has gaps.

`mount_virtual_fs / unmount_virtual_fs`:

- Uses `--bind` for `/dev` and `/dev/pts`, `proc`, `sysfs`, `tmpfs` for `/run` — standard and mostly correct.
- `unmount_virtual_fs` runs `umount` without `-R` or recursive handling and ignores errors — this can leave stale mounts.
- Cleanup on failure is attempted via `finally`, which is good.

`run_post_install_hooks`:

- Scans build.sh for the string post_install() — fragile heuristic (could false-positive on comments or other functions).
- Copies the entire packages_dir into the target (/tmp/installer-packages) — works but bloats the target temporarily.
- Runs hooks with a simple source ... && post_install — matches the build system's custom style.
- Does **not** fail the entire install on hook errors (pragmatic, since some hooks may expect running services).

Issues:

- No /dev/pts mount in some paths (minor).
- No mount --make-rprivate or careful bind-mount cleanup (can cause issues on unmount in nested setups).
- No network or DBus forwarding if hooks need it.

Verdict: Functionally correct for LFS-style post-install, but improve unmount robustness (use umount -R -l as fallback) and make hook detection more reliable (parse bash or require explicit metadata in package.yml).

4. User Creation Security

Reasonable but basic.

- Passwords are passed in plain text to chpasswd (standard, hashes on the target).
- useradd -m -G wheel,audio,video,... — good default groups for desktop use.
- Creates wheel group and enables %wheel in sudoers if the comment is present — works but brittle (depends on the exact sudoers template from packages).
- Root password expiry disabled (passwd -x 99999).
- No password quality checking, no passwd --status verification.

Security notes:

- Passwords are held in memory only during the TUI (good).
- Running as root installer is inherent.
- For desktop, the groups are sensible; consider adding plugdev, storage, etc., if needed.

Verdict: Acceptable for v1. Consider adding an option for SSH key setup later.

5. GRUB Installation

Mostly correct but requires careful testing.

- EFI detection via `is_efi()` — good.
- EFI: `grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=InterGenOS` — standard and correct.
- BIOS: `grub-install --target=i386-pc <disk>` — correct for GPT + `bios_grub` partition.
- Then `grub-mkconfig -o /boot/grub/grub.cfg` inside `chroot` — good.
- `GRUB_CMDLINE_LINUX` set with `root=UUID=...` — excellent.
- `GRUB_DISABLE_OS_PROBER=true` — sensible for a single-OS install.

Risks:

- EFI variables may not be accessible in some live/rescue environments.
- If the live environment is booted in BIOS mode but the target disk is for EFI, mismatch can occur (rare if user follows prompts).
- No `--removable` for EFI (can be useful for some systems).

Verdict: Correct invocations. Test thoroughly in both EFI and BIOS VMs.

6. TUI Robustness (ncurses)

The TUI is simple and functional but minimal.

- Uses `curses.wrapper` — good.
- Basic color pairs, header, message, prompt, `yes_no`.
- Password input uses `noecho` — correct.
- No handling for terminal resize (`SIGWINCH` / `KEY_RESIZE`).
- Input validation is basic (try/except for disk number); no bounds checking on some prompts.
- Group selection screen is placeholder ("just accept defaults") — needs real toggle logic.
- No progress bar for package install beyond a simple text bar (acceptable).

Verdict: Works for a minimal installer but will break or look bad on resize. Add `KEY_RESIZE` handling and `resizeterm` / redraw logic. Improve input validation and make group selection interactive.

7. General Safety

High-risk areas:

- Destructive partitioning with minimal confirmation (see #1).
- Package extraction relies on pkm's PackageInstaller — which (from previous review) uses plain tar -xzf without strong path sanitization. Since archives come from your trusted build system, risk is low in normal use, but still worth hardening (use Python tarfile with filter='data' or explicit path validation).
- No rollback on partial failure (e.g., if packages fail midway, the target is left in inconsistent state).
- Runs entirely as root with no privilege separation.
- No logging of the entire install process to a file (very useful for debugging failed installs).

Positives:

- Reuses pkm with --root — consistent with build system.
- Unmount at the end.
- Clear warnings about data loss.

Overall Assessment & Recommendations

Forge has a clean architecture (backend modules are nicely separated, TUI is straightforward). It successfully orchestrates the complex pieces needed for a from-source distro installer. However, because it performs **irreversible destructive actions**, it needs stronger safety rails before real hardware testing.

Priority fixes (must-do before first real install):

1. **Strengthen disk confirmation** — Add a review screen showing exact parted commands + require typing the disk path or "ERASE".
2. **Harden archive extraction** in pkm (and thus Forge) — Switch to safe tar handling with path validation to prevent any theoretical tar-slip issues.
3. **Improve unmount robustness** — Use umount -R -l as fallback and verify nothing is busy.
4. **Add basic logging** — Write all steps, commands, and errors to /var/log/forge-install.log on both host and target.

5. **Make TUI more robust** — Handle KEY_RESIZE, improve group selection, add better validation.

Nice-to-have / future:

- Optional swap partition or LVM.
- Encryption support (LUKS).
- More flexible partitioning (manual or guided with separate /home).
- Dry-run mode for the entire installer.
- Better hook detection (add "post_install": true to package.yml).
- Progress estimation and error recovery.

This installer is very close to being usable for development/testing in VMs. Once the safety gaps are closed, it should provide a smooth "forge your own system" experience that matches the project's philosophy.

Test it aggressively in VMs first (QEMU with multiple virtual disks, both EFI and BIOS). Simulate failures (bad archives, interrupted install) and verify cleanup.

If you'd like, I can suggest concrete code patches for the confirmation screen, safe tar extraction helper, or improved unmount logic. Or share specific parts (e.g., a sample build.sh with post_install) for deeper integration review. Great work tying everything together — with a few hardening passes, this will be solid.